

Artificial Intelligence (Spring 2024)

AI Week 5

**[Dataset Exploration Techniques,
PairPlot Visualization, Correlation Matrix]**



Contents

- 1. Dataset Exploration Techniques**
- 2. PairPlot Visualization**
- 3. Correlation Matrix**

Dataset Exploration Techniques

1. info() Method

- **info()** This function is particularly useful during exploratory analysis, offering a quick and informative overview of the dataset.
- Leveraging ``dataframe.info()`` is an efficient way to gain insights into the structure and characteristics of the data, making it an essential step in the data analysis workflow.

Dataset Exploration Techniques

1. info() Method

It provides essential details such as the

1. Total **number of non-null** values
2. **Data types** of each column
3. Memory usage

This summary is beneficial for

1. Quickly assessing the **completeness of the dataset**
2. Identifying **potential missing values**
3. Understanding the **overall data types** present in the DataFrame.

Dataset Exploration Techniques

1. info() Method

Load Dataset

```
# importing pandas as pd
import pandas as pd

# Creating the dataframe
df = pd.read_csv("nba.csv")

# Print the dataframe
df
```

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
...	...	...	...	...	...	...	...	...	...
453	Shelvin Mack	Utah Jazz	8.0	PG	26.0	6-3	203.0	Butler	2433333.0
454	Raul Neto	Utah Jazz	25.0	PG	24.0	6-1	179.0	NaN	900000.0
455	Tibor Pleiss	Utah Jazz	21.0	C	26.0	7-3	256.0	NaN	2900000.0
456	Jeff Withey	Utah Jazz	24.0	C	26.0	7-0	231.0	Kansas	947276.0
457	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

458 rows × 9 columns

Dataset Exploration Techniques

1. info() Method

```
# to print the full summary  
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 458 entries, 0 to 457  
Data columns (total 9 columns):  
Name           457 non-null object  
Team           457 non-null object  
Number         457 non-null float64  
Position       457 non-null object  
Age            457 non-null float64  
Height         457 non-null object  
Weight         457 non-null float64  
College        373 non-null object  
Salary         446 non-null float64  
dtypes: float64(4), object(5)  
memory usage: 32.3+ KB
```

Dataset Exploration Techniques

2. describe() Method

- The **describe()** method returns description of the data in the DataFrame.
- If the DataFrame contains numerical data, the description contains these information for each column

Dataset Exploration Techniques

2. describe() Method

count - The number of not-empty values.

mean - The average (mean) value.

std - The standard deviation.

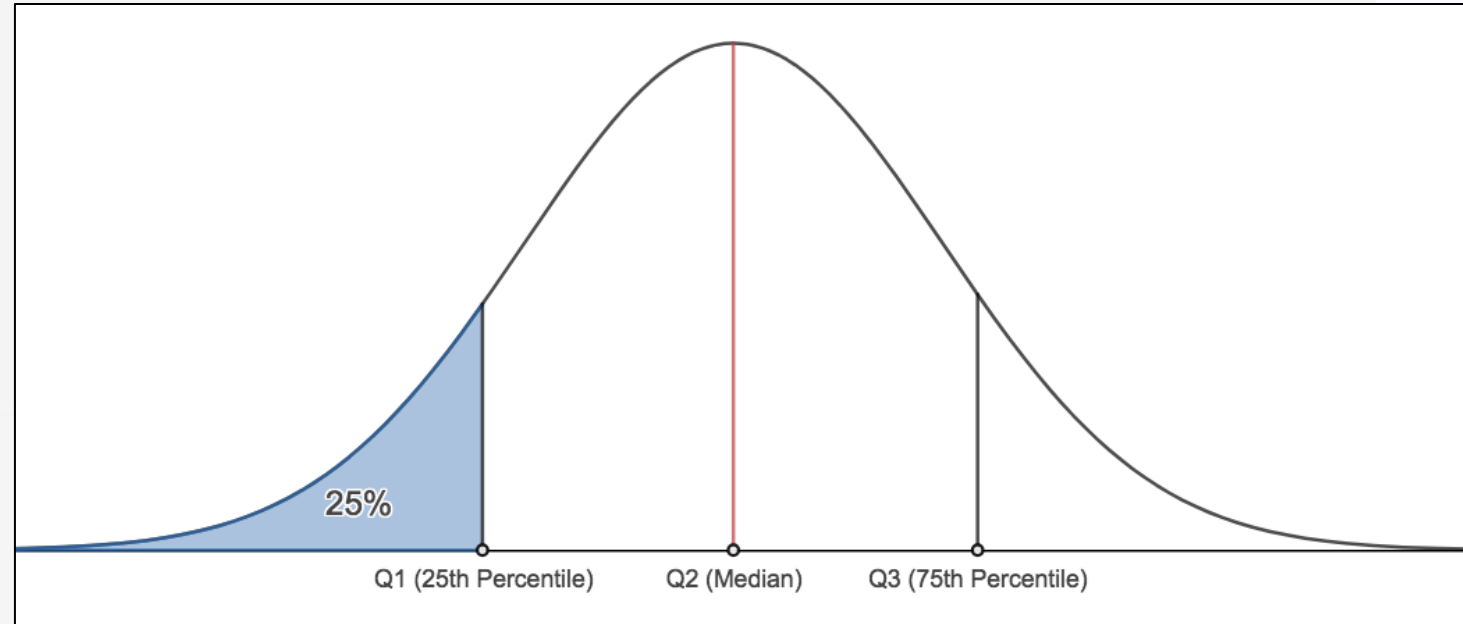
min - the minimum value.

25% - The 25% percentile*.

50% - The 50% percentile*.

75% - The 75% percentile*.

max - the maximum value.



Dataset Exploration Techniques

2. describe() Method

```
import pandas as pd

data = [[10, 18, 11], [13, 15, 8], [9, 20, 3]]

df = pd.DataFrame(data)

print(df.describe())
```

	0	1	2
count	3.000000	3.000000	3.000000
mean	10.666667	17.666667	7.333333
std	2.081666	2.516611	4.041452
min	9.000000	15.000000	3.000000
25%	9.500000	16.500000	5.500000
50%	10.000000	18.000000	8.000000
75%	11.500000	19.000000	9.500000
max	13.000000	20.000000	11.000000

Dataset Exploration Techniques

3. head() & tail() Methods

The **head()** method returns the **first n rows**.

```
print(df.head())
```

#	col_0	col_1
# row_0	A	9
# row_1	B	8
# row_2	C	7
# row_3	D	6
# row_4	E	5

```
print(df.head(3))
```

#	col_0	col_1
# row_0	A	9
# row_1	B	8
# row_2	C	7

Dataset Exploration Techniques

3. head() & tail() Methods

The **tail()** method returns the **last n rows**.

```
print(df.tail())
```

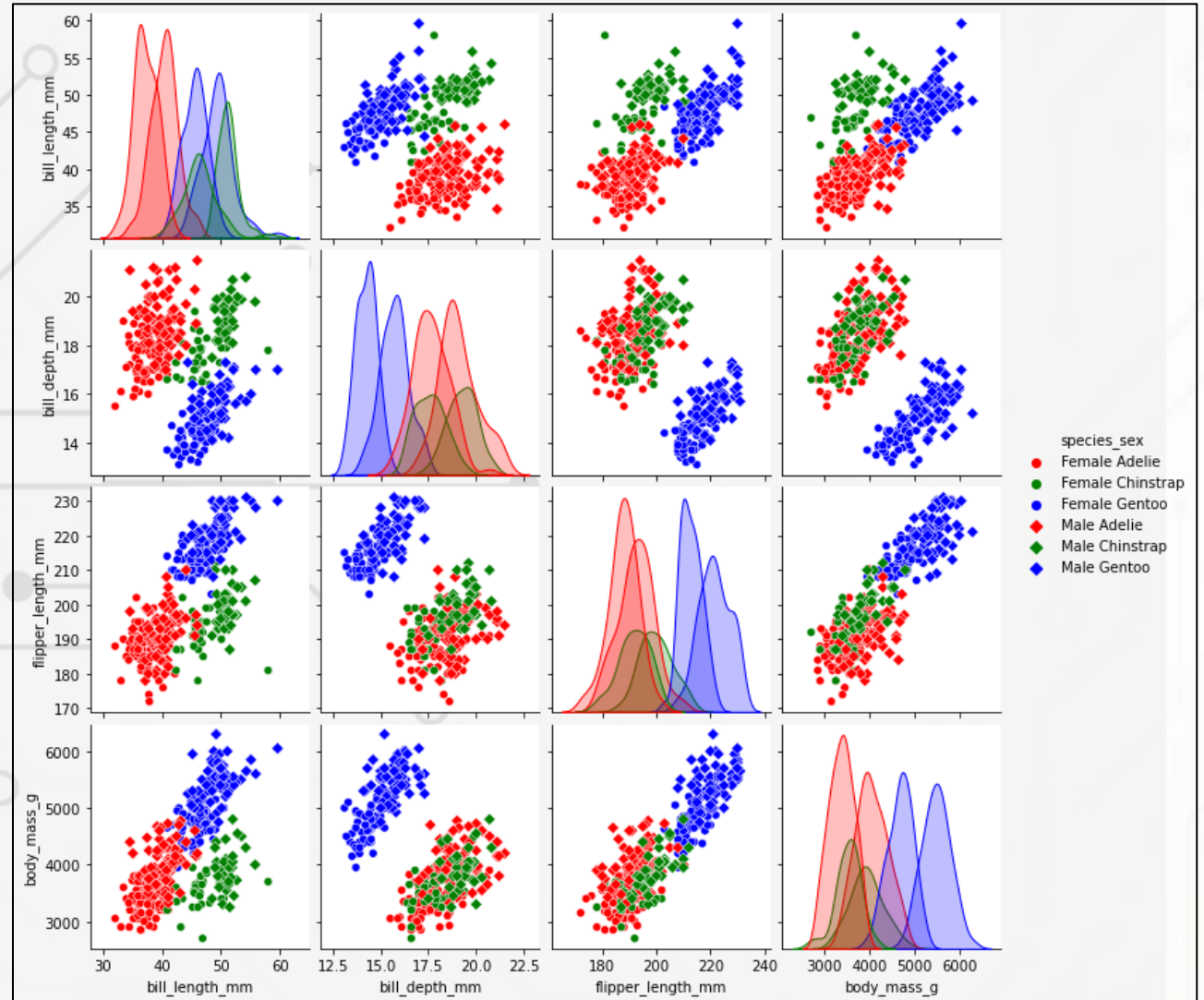
#	col_0	col_1
# row_5	F	4
# row_6	G	3
# row_7	H	2
# row_8	I	1
# row_9	J	0

```
print(df.tail(3))
```

#	col_0	col_1
# row_7	H	2
# row_8	I	1
# row_9	J	0

PairPlot Visualization

Pairplot graph in Python using the **seaborn** library.



PairPlot Visualization

A pairplot graph is a graph that plots pairwise relationships in a dataset.

For example:

- ✓ In a dataset that compares the **test scores of men and women**, it would plot the data of **men in blue** and the data of **women in red** beside each other so that the two can be compared.
- ✓ In a dataset comparing **cats with dogs**, it would plot the data of **cats in green** and **dogs in yellow**. This is so that you can easily see the relationship between cats and dogs highlighted well.

PairPlot Visualization

Iris Flowers Dataset

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica



petal

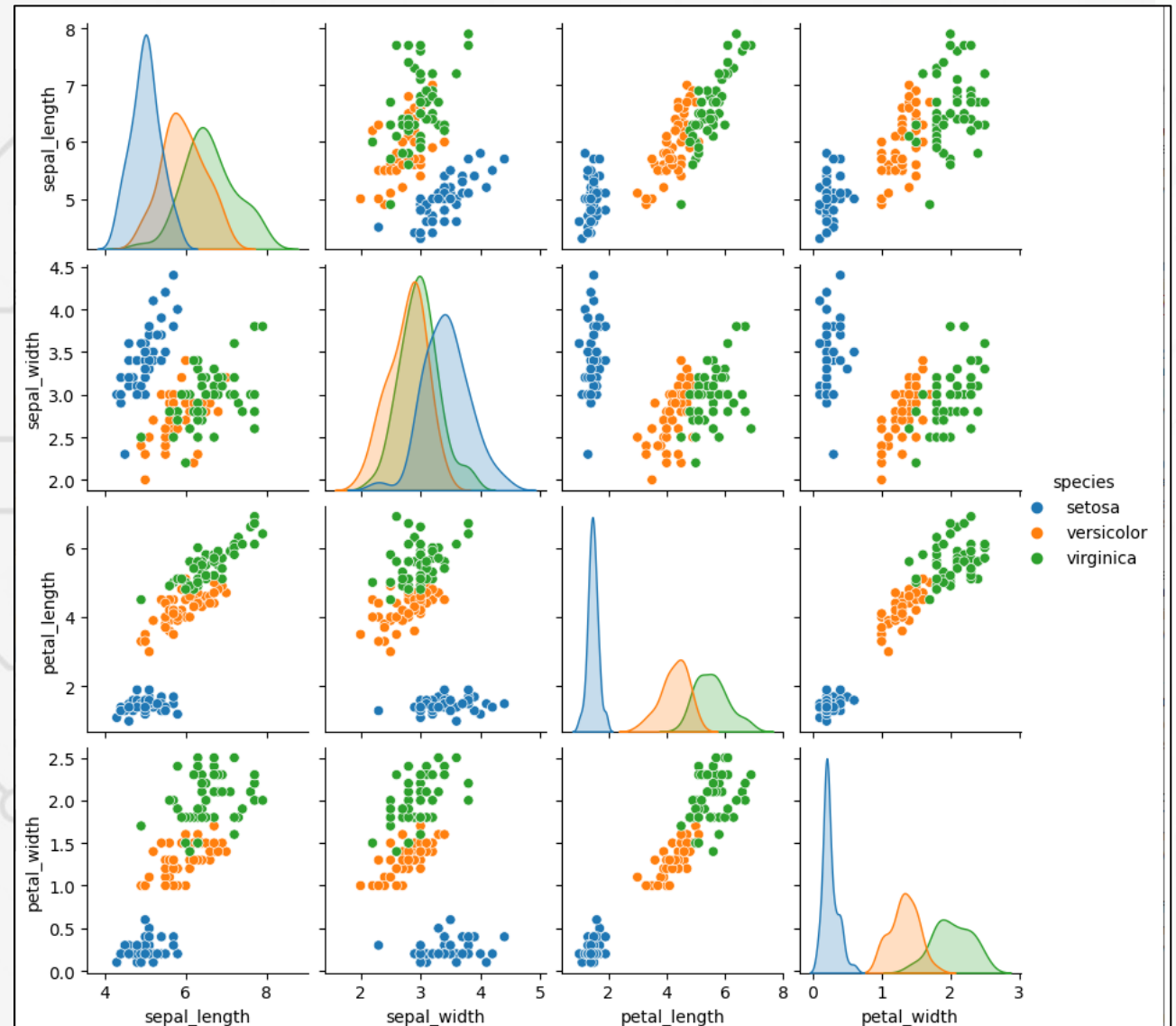
sepal

PairPlot Visualization

```
import seaborn as sns
import matplotlib.pyplot as plt

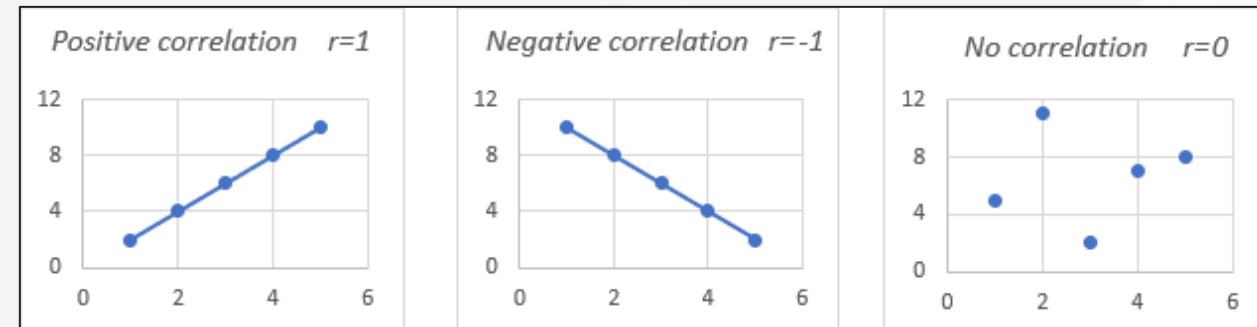
iris= sns.load_dataset('iris')

sns.pairplot(iris, hue='species')
plt.show()
```



Correlation Matrix

- The **correlation coefficient** is the relationship between two or more variables, usually represented using the symbol r , and it **ranges from -1 to +1**.
- A correlation coefficient quite close to 0, but either positive or negative, implies little or no relationship between the two variables.



Correlation Matrix

A correlation matrix is simply a table showing the correlation coefficients between variables. (The table has used data from health dataset)

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage	Hours_Work	Hours_Sleep
Duration	1	-0.17	0.00	0.89	-0.12	0.07
Average_Pulse	-0.17	1	0.79	0.02	-0.28	0.03
Max_Pulse	0.00	0.79	1	0.20	-0.27	0.09
Calorie_Burnage	0.89	0.02	0.20	1	-0.14	0.08
Hours_Work	-0.12	-0.28	-0.27	-0.14	1	-0.14
Hours_Sleep	0.07	0.03	0.09	0.08	-0.14	1

Correlation Matrix

We can use the **corr()** function in Python to create a correlation matrix.
We also use the **round()** function to round the output to two decimals:

```
import pandas as pd

full_health_data = pd.read_csv("data.csv", header=0, sep=",")

Corr_Matrix = round(full_health_data.corr(),2)

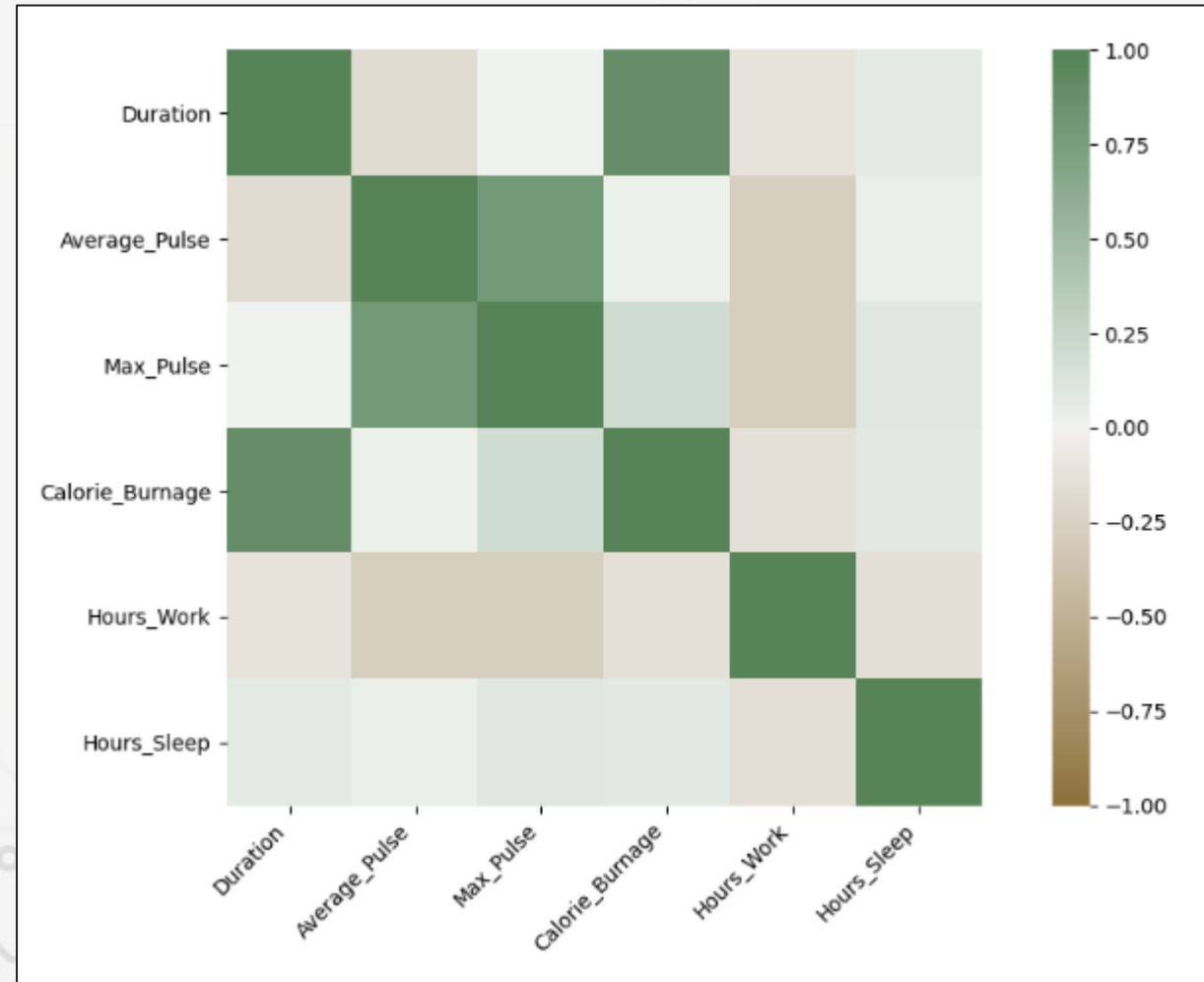
print(Corr_Matrix)
```

	Duration	Average_Pulse	Max_Pulse	Calorie_Burnage
Duration	1.00	-0.17	0.00	0.89
Average_Pulse	-0.17	1.00	0.79	0.02
Max_Pulse	0.00	0.79	1.00	0.20
Calorie_Burnage	0.89	0.02	0.20	1.00

Correlation Matrix

We can use a **Heatmap** to **Visualize the Correlation** Between Variables:

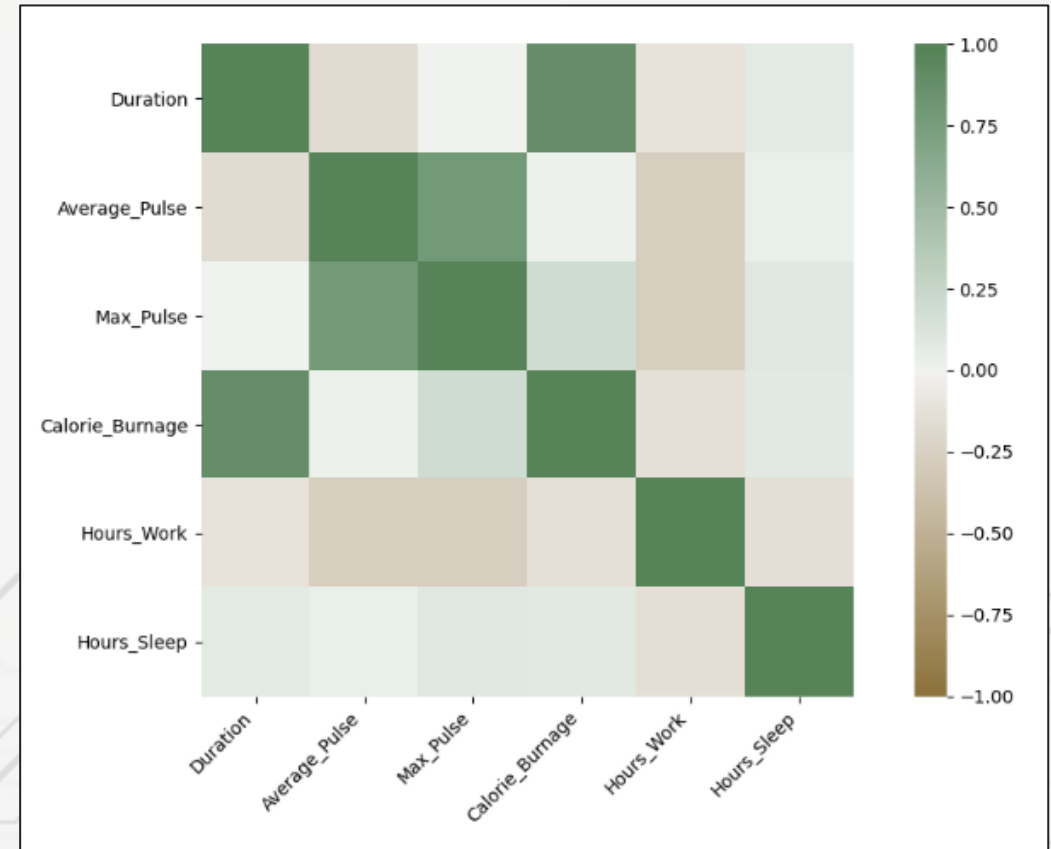
- The closer the correlation coefficient is **to 1**, the **greener** the squares get.
- The closer the correlation coefficient is **to -1**, the **browner** the squares get.

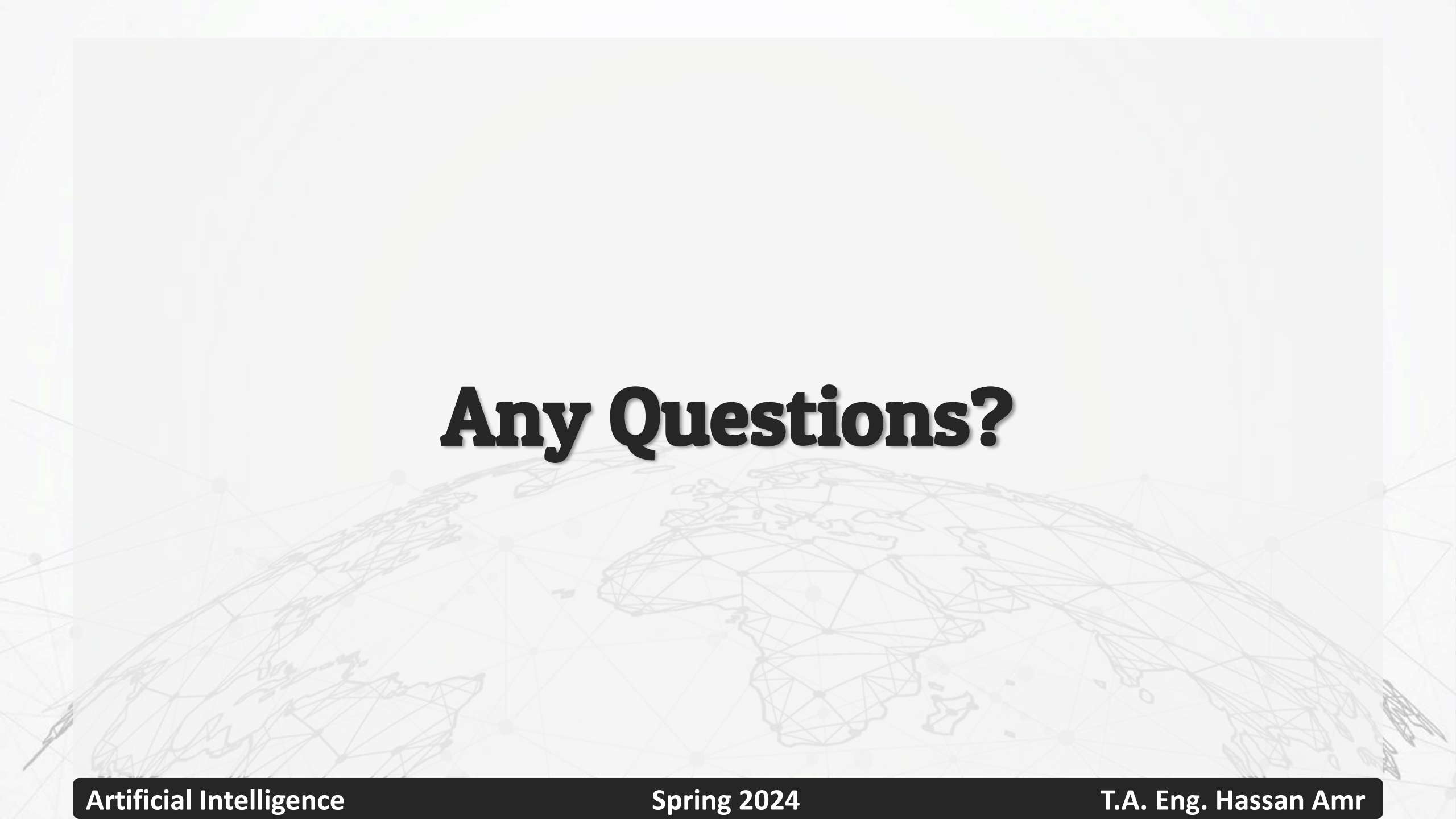


Correlation Matrix

Use Seaborn to Create a Heatmap

```
axis_corr = sns.heatmap(  
    correlation_full_health,  
    vmin=-1, vmax=1, center=0,  
    cmap=sns.diverging_palette(50, 500, n=500),  
    square=True  
)  
  
plt.show()
```



The background of the slide features a stylized, light gray globe. Overlaid on the globe is a complex network of thin gray lines connecting various points, resembling a global communication or data network. The text "Any Questions?" is centered in a large, bold, black font with a slight shadow effect.

Any Questions?